

Rising Waters – Flood Prediction System

Project Description:

Rising Waters is a machine learning-based flood prediction system developed to identify the likelihood of flood occurrence using historical weather and rainfall data. The project analyzes environmental factors such as temperature, humidity, cloud cover, and rainfall patterns to predict flood risk.

Multiple machine learning algorithms, including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost, are trained and compared to select the best-performing model. The final model is deployed using a Flask web application, providing a simple and user-friendly interface for predicting flood risk and supporting early disaster preparedness.

The application also includes an analytics page to display model performance and a contact page with emergency flood helpline information.

Scenarios

Scenario 1: Flood Risk Prediction

A disaster management officer enters the current weather and rainfall details, including temperature, humidity, cloud cover, and seasonal rainfall, into the application. The system analyzes the input and predicts whether there is a potential flood risk, helping authorities take preventive measures in advance.

Scenario 2: Weather Data Analysis

A researcher uses the application to analyze historical weather data and compare the performance of different machine learning models. By viewing the analytics page, the researcher can evaluate model accuracy and understand which algorithm performs best for flood prediction.

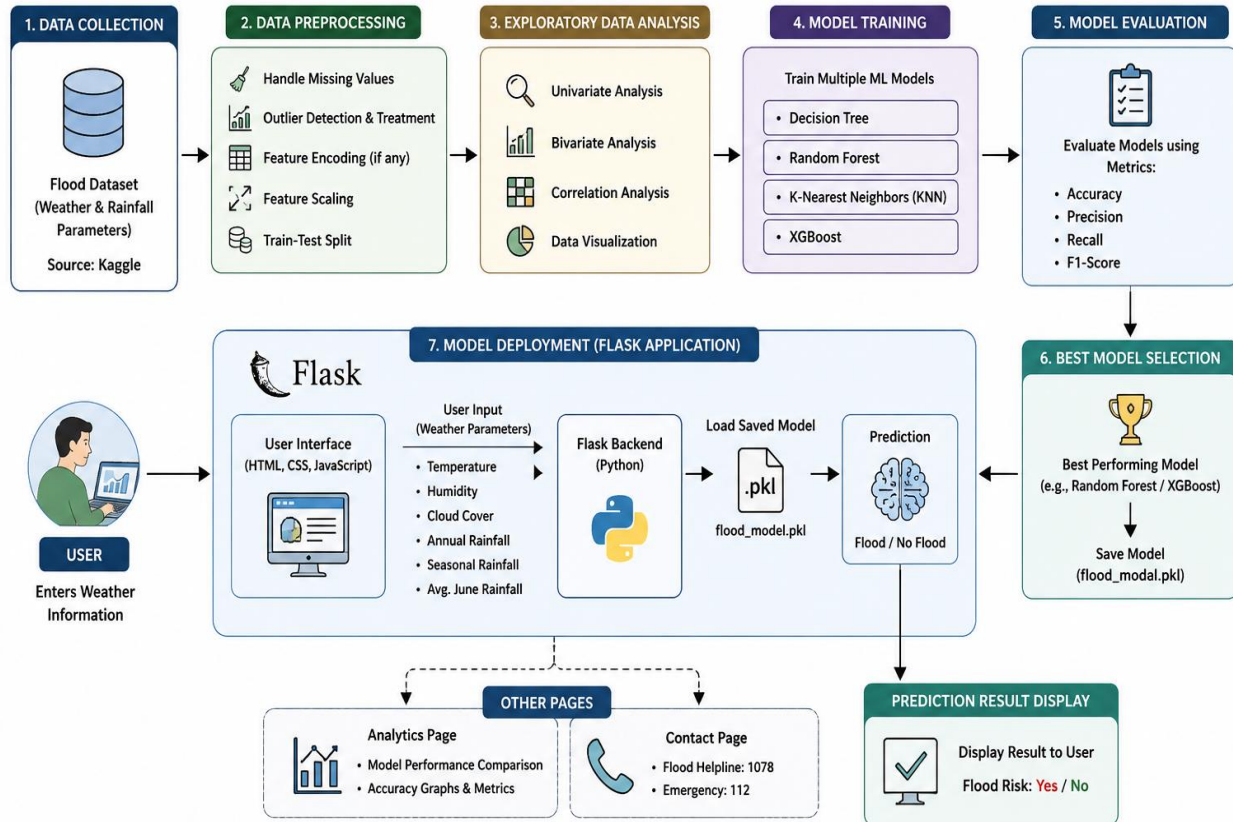
Scenario 3: Emergency Awareness

A community member accesses the flood prediction application to check the flood risk in their area. If needed, they can use the **Contact** page to find emergency flood helpline numbers, enabling quick communication with disaster management authorities during flood-related emergencies.

Technical Architecture:

Rising Waters – Flood Prediction System

Technical Architecture



Description: The **Rising Waters – Flood Prediction System** follows a machine learning architecture that collects and preprocesses historical weather and rainfall data before training multiple models, including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost. The best-performing model is saved as a Pickle (.pkl) file and integrated into a Flask web application. Users enter weather parameters through a simple web interface, and the application processes the input to generate flood risk predictions. The system also includes an Analytics page to display model performance and a Contact page with emergency flood helpline information, providing a complete and user-friendly flood prediction solution.

Pre-requisites:

- **Operating System:** Windows 10/11, macOS, or Linux
- **Python:** Version 3.10 or above
- **Anaconda Navigator** (optional, for Jupyter Notebook)
- **Jupyter Notebook** for data analysis and model development

- **Visual Studio Code (VS Code)** for Flask application development
- **Flask** for building the web application
- **Python Libraries:** Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, XGBoost, and Pickle
- **Git and GitHub** for version control and project hosting
- **PythonAnywhere** (or any Flask-supported hosting platform) for deployment
- **Flood Prediction Dataset** downloaded from Kaggle

Project Workflow:

Milestone 1: Project Planning and Data Collection

- **Activity 1.1:** Define the project objective
- **Activity 1.2:** Collect the flood prediction dataset
- **Activity 1.3:** Organize the project structure
- **Activity 1.4:** Load the dataset into Jupyter Notebook

Milestone 2: Data Analysis and Visualization

- **Activity 2.1:** Import required Python libraries
- **Activity 2.2:** Explore the dataset structure
- **Activity 2.3:** Perform univariate analysis
- **Activity 2.4:** Perform multivariate analysis
- **Activity 2.5:** Conduct descriptive statistical analysis

Milestone 3: Data Preprocessing

- **Activity 3.1:** Handle missing values
- **Activity 3.2:** Detect and treat outliers
- **Activity 3.3:** Prepare features and target variable
- **Activity 3.4:** Split the dataset into training and testing sets
- **Activity 3.5:** Apply feature scaling

Milestone 4: Model Development and Evaluation

- **Activity 4.1:** Train the Decision Tree model
- **Activity 4.2:** Train the Random Forest model

- **Activity 4.3:** Train the K-Nearest Neighbors (KNN) model
- **Activity 4.4:** Train the XGBoost model
- **Activity 4.5:** Compare model performance
- **Activity 4.6:** Select and save the best-performing model

Milestone 5: Application Development and Deployment

- **Activity 5.1:** Design the web interface using HTML and CSS
- **Activity 5.2:** Develop the Flask application
- **Activity 5.3:** Integrate the trained model
- **Activity 5.4:** Test the application
- **Activity 5.5:** Deploy the application on PythonAnywhere

Milestone 6: Conclusion

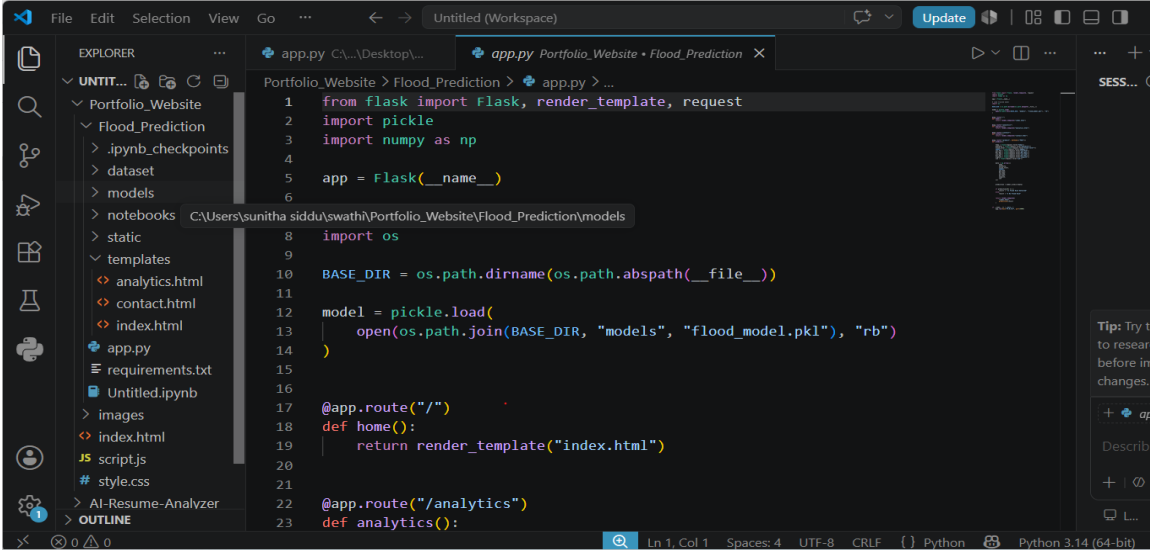
Milestone 1: Project Planning and Data Collection

Objective:

The first milestone focuses on planning the project and collecting the dataset required for flood prediction. During this phase, the project objectives are defined, the project structure is organized, and the flood prediction dataset is collected and loaded into the Jupyter Notebook for further analysis and model development.

Activity 1.1: Define the Project Objective

The objective of the project is to develop a machine learning-based flood prediction system that predicts flood risk using historical weather and rainfall data. The system is designed to assist users by providing accurate flood risk predictions through a web-based application.



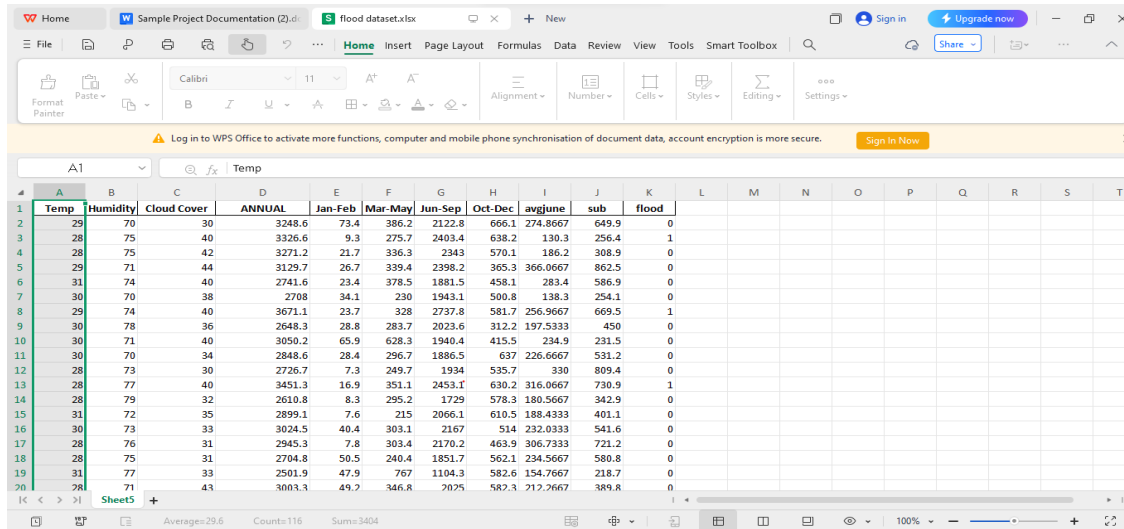
The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure for 'Portfolio_Website' with subfolders like 'Flood_Prediction', 'dataset', 'models', 'notebooks', 'static', 'templates', and 'images'. The code editor shows the following Python code:

```

1 from flask import Flask, render_template, request
2 import pickle
3 import numpy as np
4
5 app = Flask(__name__)
6
7
8 import os
9
10 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
11
12 model = pickle.load(
13     open(os.path.join(BASE_DIR, "models", "flood_model.pkl"), "rb")
14 )
15
16
17 @app.route("/")
18 def home():
19     return render_template("index.html")
20
21
22 @app.route("/analytics")
23 def analytics():
  
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

The flood prediction dataset is downloaded from the Kaggle repository. The dataset contains historical weather and rainfall parameters that are used to train and evaluate the machine learning models.



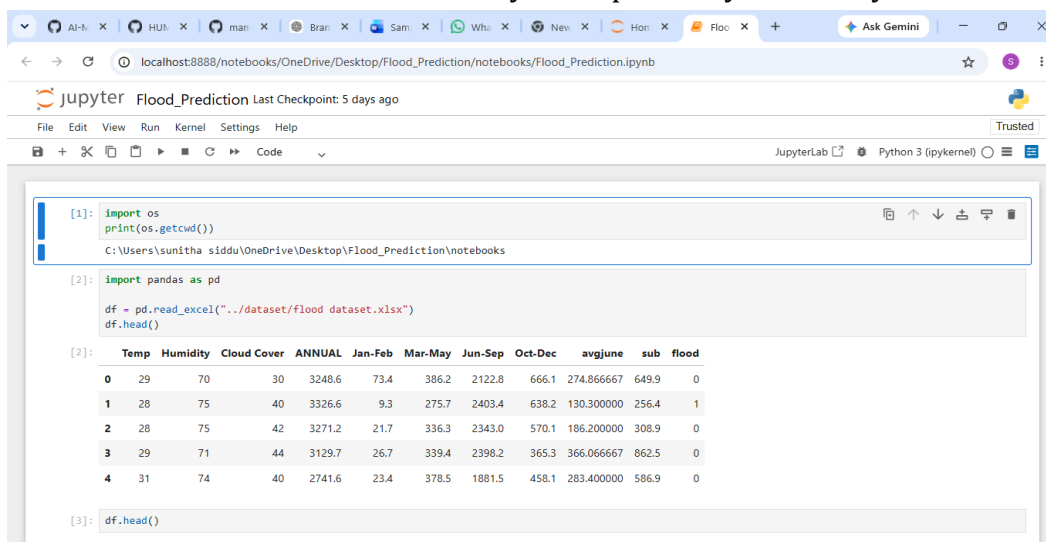
	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
1	29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.86667	649.9	0
2	28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.3	256.4	1
3	28	75	42	3271.2	21.7	336.3	2343	570.1	186.2	308.9	0
4	29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.06667	862.5	0
5	31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.4	586.9	0
6	30	70	38	2708	34.1	230	1943.1	500.8	138.3	254.1	0
7	29	74	40	3671.1	23.7	328	2737.8	581.7	256.9667	669.5	1
8	30	78	36	2648.3	28.8	283.7	2023.6	312.2	197.5333	450	0
9	30	71	40	3050.2	65.9	628.3	1940.4	415.5	234.9	231.5	0
10	30	70	34	2848.6	28.4	296.7	1886.5	637	226.6667	531.2	0
11	28	73	30	2726.7	7.3	249.7	1934	535.7	330	809.4	0
12	28	77	40	3451.3	16.9	351.1	2453.1	630.2	316.0667	730.9	1
13	28	79	32	2610.8	8.3	295.2	1729	578.3	180.5667	342.9	0
14	31	72	35	2899.1	7.6	215	2066.1	610.5	188.4333	401.1	0
15	30	73	33	3024.5	40.4	303.1	2167	514	232.0333	541.6	0
16	28	76	31	2945.3	7.8	303.4	2170.2	463.9	306.7333	721.2	0
17	28	75	31	2704.8	50.5	240.4	1851.7	562.1	234.5667	580.8	0
18	31	77	33	2501.9	47.9	767	1104.3	582.6	154.7667	218.7	0
19	28	71	43	3003.3	49.2	346.8	2025	582.3	212.2667	389.8	0

Activity 1.3: Organize the Project Structure

The project directory is organized into separate folders for datasets, notebooks, models, templates, static files, and application files. This structure helps manage the project efficiently during development and deployment.

Activity 1.4: Set Up the Development Environment

The dataset is loaded into Jupyter Notebook using the Pandas library. After loading, the dataset is verified to ensure it is ready for exploratory data analysis and model development.



```
[1]: import os
print(os.getcwd())
C:\Users\sunitha\OneDrive\Desktop\Flood_Prediction\notebooks

[2]: import pandas as pd

df = pd.read_excel("../dataset/flood dataset.xlsx")
df.head()

[2]:
```

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
0	29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.866667	649.9	0
1	28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.300000	256.4	1
2	28	75	42	3271.2	21.7	336.3	2343.0	570.1	186.200000	308.9	0
3	29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.066667	862.5	0
4	31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.400000	586.9	0

```
[3]: df.head()
```

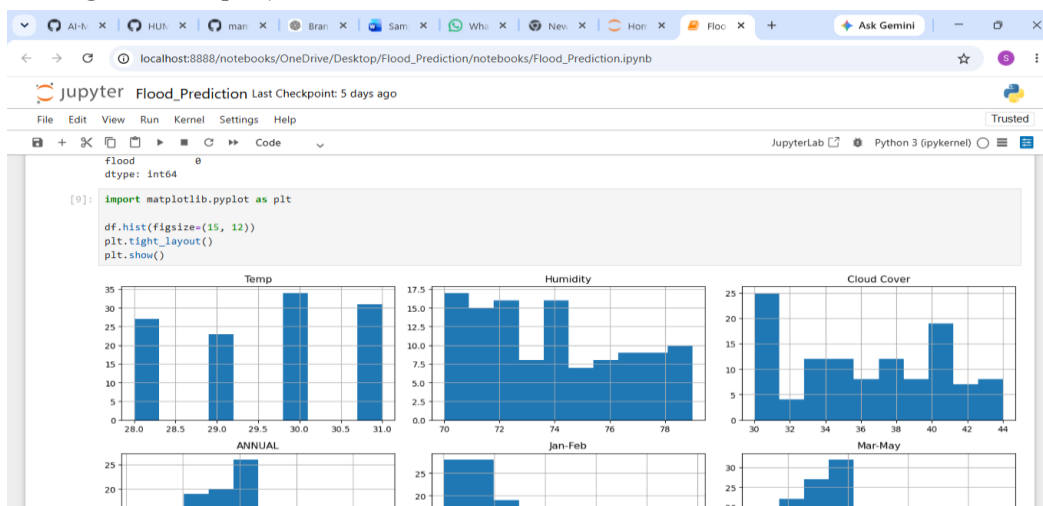
Milestone 2: Data Analysis and Visualization

Objective:

The objective of this milestone is to analyze and visualize the flood prediction dataset to understand its structure, identify the relationships among features, and gain meaningful insights. This analysis helps in understanding the data distribution and prepares the dataset for effective preprocessing and machine learning model development.

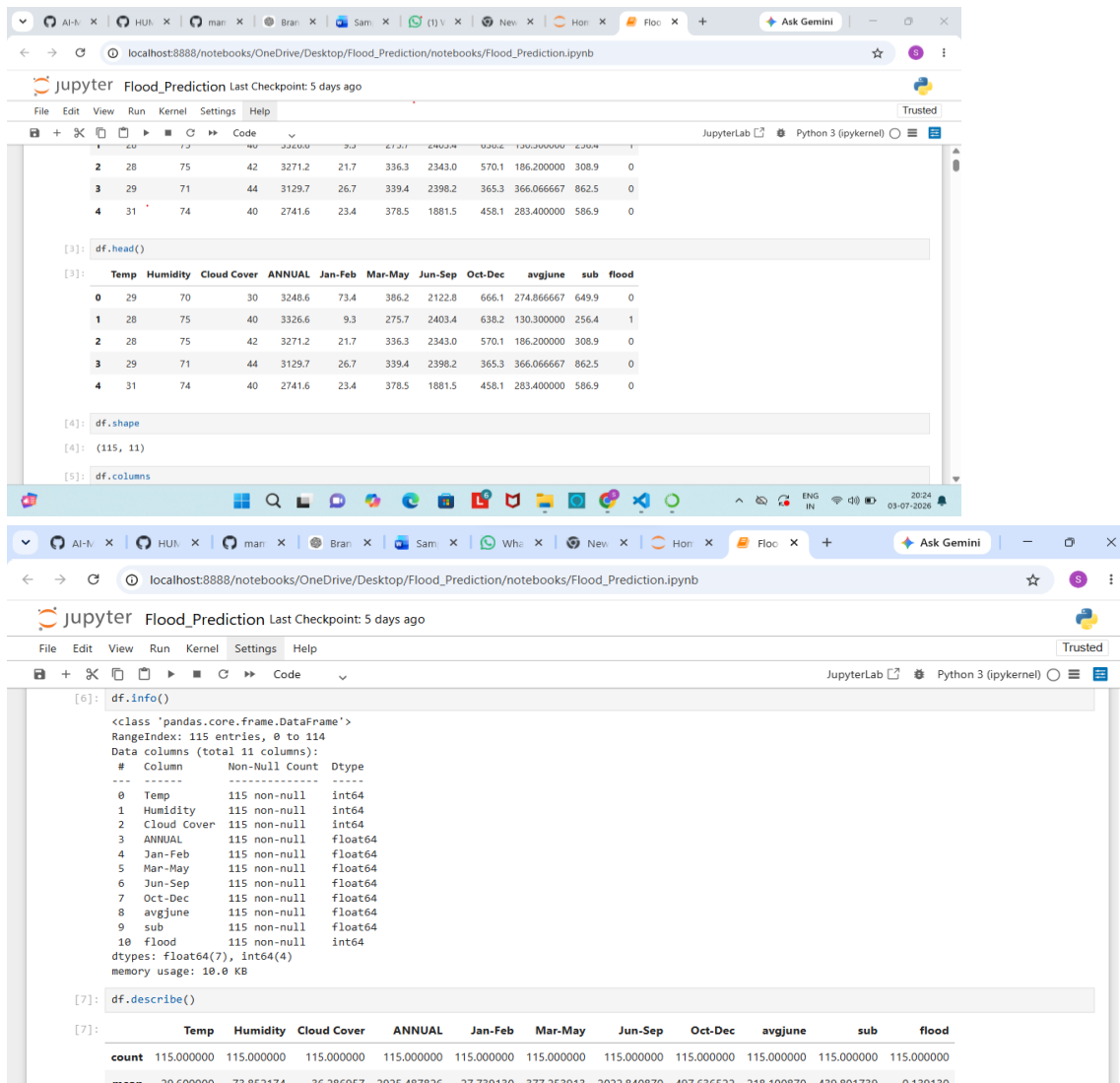
Activity 2.1: Import Required Python Libraries

The required Python libraries, including Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn, are imported to perform data manipulation, visualization, and machine learning tasks throughout the project.



Activity 2.2: Explore the Dataset

The dataset is examined to understand its structure, data types, dimensions, column names, and target variable. Functions such as `head()`, `info()`, `shape`, and `describe()` are used to gain an overview of the dataset.



JupyterLab Flood_Prediction Last Checkpoint: 5 days ago

File Edit View Run Kernel Settings Help Trusted

Code

```
[1]: df.head()
```

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
0	29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.866667	649.9	0
1	28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.300000	256.4	1
2	28	75	42	3271.2	21.7	336.3	2343.0	570.1	186.200000	308.9	0
3	29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.066667	862.5	0
4	31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.400000	586.9	0

```
[3]: df.shape
[4]: (115, 11)
[5]: df.columns
```

JupyterLab Flood_Prediction Last Checkpoint: 5 days ago

File Edit View Run Kernel Settings Help Trusted

Code

```
[6]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 115 entries, 0 to 114
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Temp        115 non-null    int64
1   Humidity    115 non-null    int64
2   Cloud Cover 115 non-null    int64
3   ANNUAL      115 non-null    float64
4   Jan-Feb     115 non-null    float64
5   Mar-May     115 non-null    float64
6   Jun-Sep     115 non-null    float64
7   Oct-Dec     115 non-null    float64
8   avgjune     115 non-null    float64
9   sub         115 non-null    float64
10  flood       115 non-null    int64
dtypes: float64(7), int64(4)
memory usage: 18.8 KB
```

```
[7]: df.describe()
```

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
count	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000
mean	29.600000	73.852174	36.286957	2925.487826	27.739130	377.253913	2022.840870	497.636522	218.100870	439.801739	0.139130

Activity 2.3: Perform Univariate Analysis

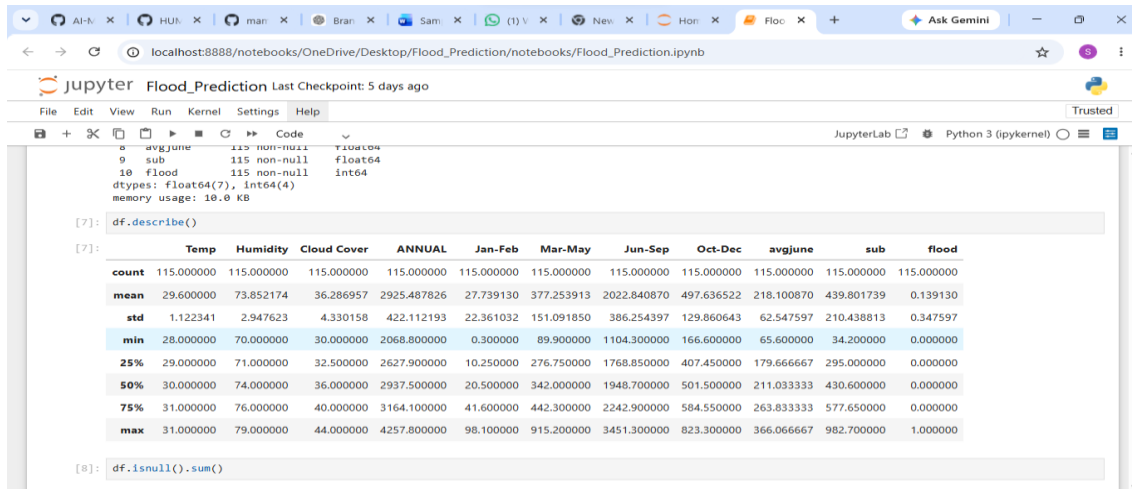
Univariate analysis is carried out to study the distribution of individual variables using plots such as histograms and count plots. This helps identify the characteristics of each feature.

Activity 2.4: Perform Multivariate Analysis

Multivariate analysis is performed to understand the relationships between multiple variables. Correlation analysis and pair plots help identify patterns and dependencies among the features.

Activity 2.5: Conduct Descriptive Statistical Analysis

Descriptive statistics are calculated to summarize the dataset using measures such as mean, median, standard deviation, minimum, and maximum values. These statistics provide insights into the overall characteristics of the data.



```

df.describe()

df.isnull().sum()
  
```

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
count	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000
mean	29.600000	73.852174	36.286957	2925.487826	27.739130	377.253913	2022.840870	497.636522	218.100870	439.801739	0.139130
std	1.122341	2.947623	4.330158	422.112193	22.361032	151.091850	386.254397	129.860643	62.547597	210.438813	0.347597
min	28.000000	70.000000	30.000000	2068.800000	0.300000	89.900000	1104.300000	166.600000	65.600000	34.200000	0.000000
25%	29.000000	71.000000	32.500000	2627.900000	10.250000	276.750000	1768.850000	407.450000	179.666667	295.000000	0.000000
50%	30.000000	74.000000	36.000000	2937.500000	20.500000	342.000000	1948.700000	501.500000	211.033333	430.600000	0.000000
75%	31.000000	76.000000	40.000000	3164.100000	41.600000	442.300000	2242.900000	584.550000	263.833333	577.650000	0.000000
max	31.000000	79.000000	44.000000	4257.800000	98.100000	915.200000	3451.300000	823.300000	366.066667	982.700000	1.000000

```

df.isnull().sum()
  
```

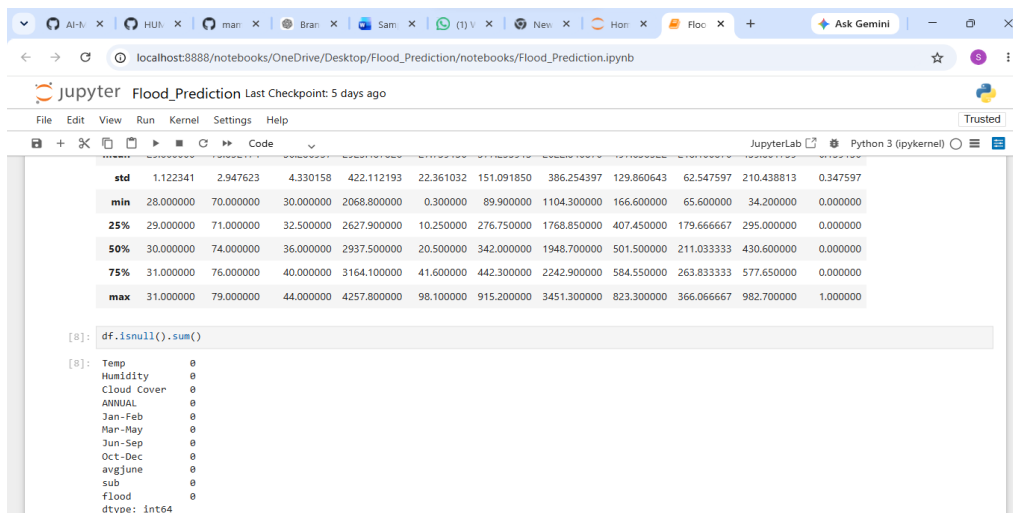
Milestone 3: Data Preprocessing

Objective:

The objective of this milestone is to preprocess the flood prediction dataset by improving data quality and preparing it for machine learning. This includes handling missing values, treating outliers, splitting the dataset into training and testing sets, and applying feature scaling to improve model performance.

Activity 3.1: Handle Missing Values

The dataset is checked for missing values using Pandas functions. Since the dataset contains no missing values, no additional data cleaning is required.



```

df.isnull().sum()
  
```

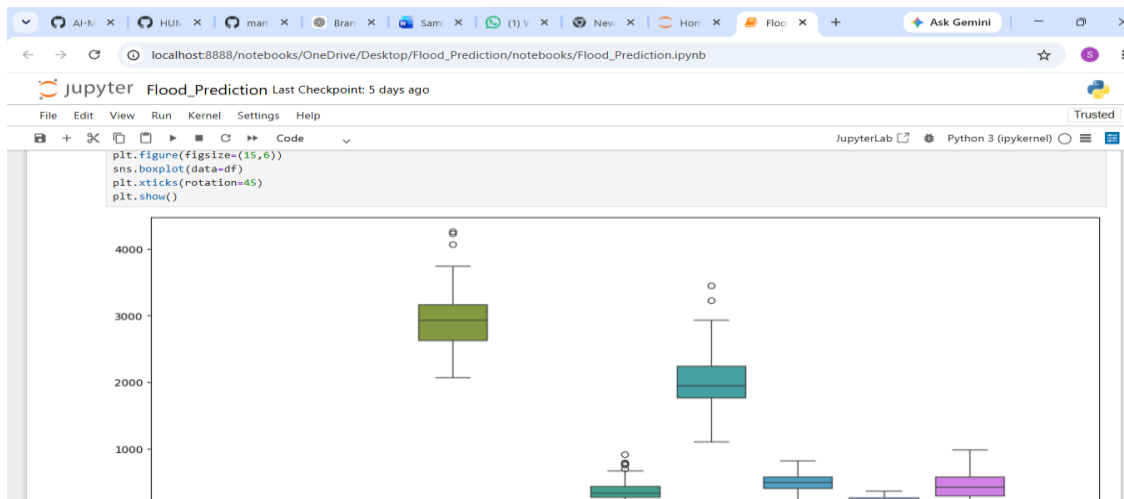
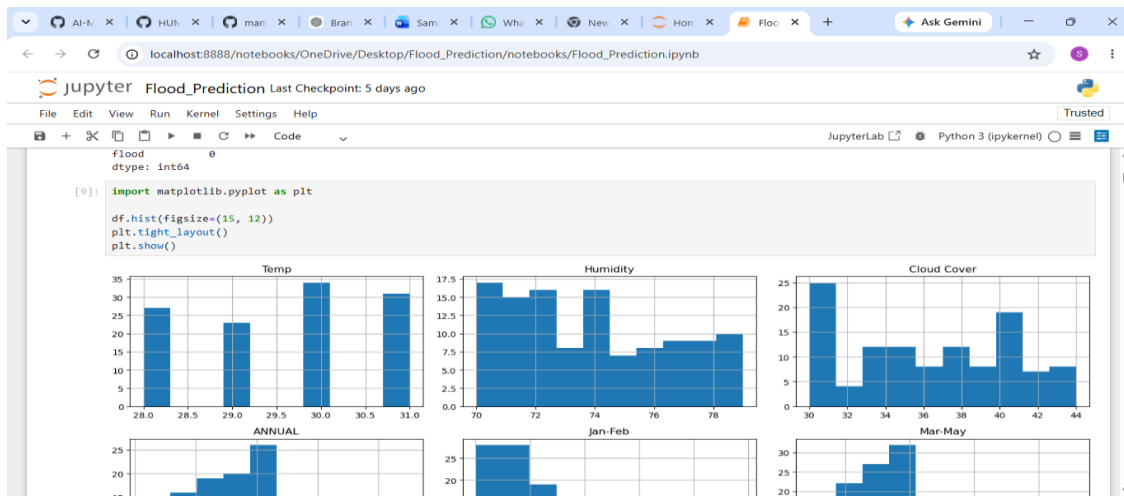
```

Temp      0
Humidity  0
Cloud Cover  0
ANNUAL    0
Jan-Feb   0
Mar-May   0
Jun-Sep   0
Oct-Dec   0
avgjune   0
sub       0
flood     0
dtype: int64
  
```

Activity 3.2: Detect and Treat Outliers

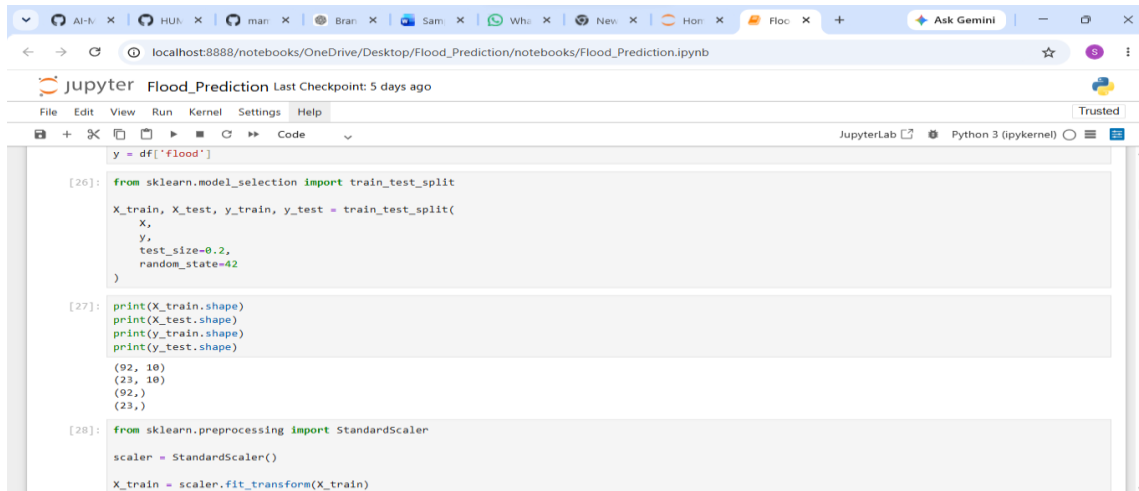
Outliers are identified using box plots and analyzed to determine their impact on the

dataset. Appropriate preprocessing techniques are applied where necessary to minimize their effect on model performance.



Activity 3.3: Split the Dataset and Apply Feature Scaling

The dataset is divided into training and testing sets using `train_test_split()`. Feature scaling is then performed using `StandardScaler` to normalize the numerical features before training the machine learning models.



```

y = df['Flood']

[26]: from sklearn.model_selection import train_test_split
      X_train, X_test, y_train, y_test = train_test_split(
      X,
      y,
      test_size=0.2,
      random_state=42
      )

[27]: print(X_train.shape)
      print(X_test.shape)
      print(y_train.shape)
      print(y_test.shape)

      (92, 10)
      (23, 10)
      (92,)
      (23,)

[28]: from sklearn.preprocessing import StandardScaler
      scaler = StandardScaler()
      X_train = scaler.fit_transform(X_train)
  
```

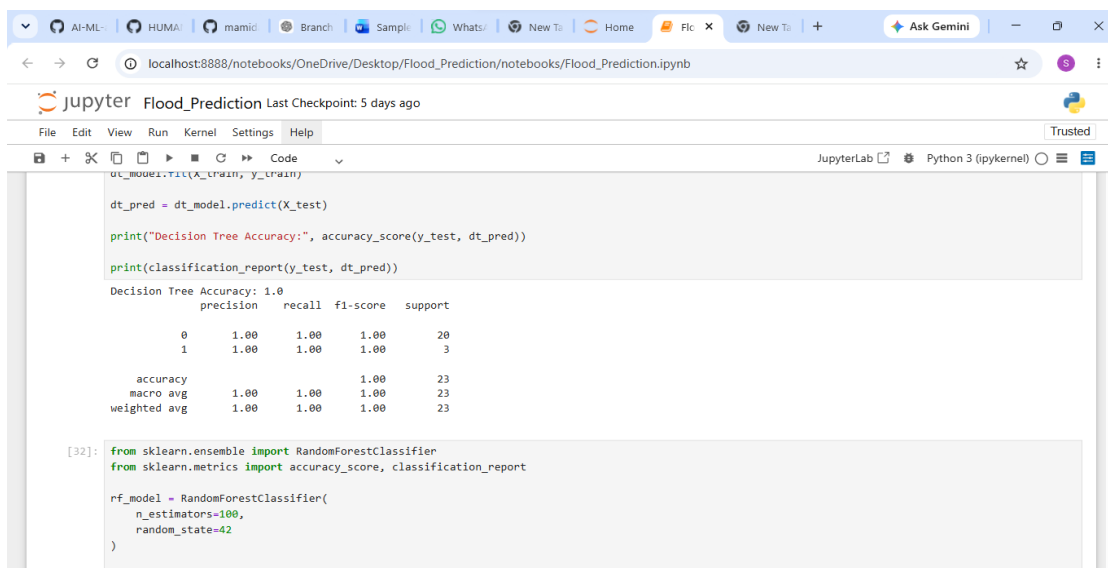
Milestone 4: Model Development and Evaluation

Objective:

The objective of this milestone is to build, train, and evaluate multiple machine learning models for flood prediction. The performance of each model is compared using appropriate evaluation metrics, and the best-performing model is selected and saved for deployment in the Flask web application.

Activity 4.1: Train Machine Learning Models

The flood prediction dataset is used to train multiple machine learning algorithms, including **Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost**. Each model learns patterns from the training data to predict flood risk.



```

dt_model.fit(X_train, y_train)

dt_pred = dt_model.predict(X_test)

print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))

print(classification_report(y_test, dt_pred))

Decision Tree Accuracy: 1.0
precision    recall  f1-score   support

      0       1.00      1.00      1.00        20
      1       1.00      1.00      1.00         3

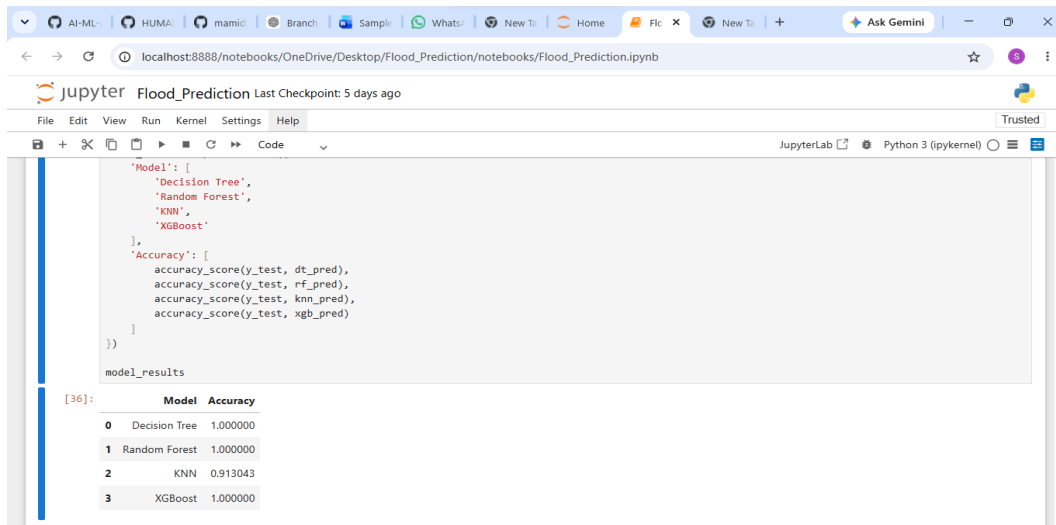
 accuracy          1.00          1.00          1.00         23
  macro avg          1.00          1.00          1.00         23
 weighted avg          1.00          1.00          1.00         23

[32]: from sklearn.ensemble import RandomForestClassifier
      from sklearn.metrics import accuracy_score, classification_report

      rf_model = RandomForestClassifier(
      n_estimators=100,
      random_state=42
      )
  
```

Activity 4.2: Evaluate and Compare Model Performance

The trained models are evaluated using performance metrics such as **Accuracy, Precision, Recall, and F1-Score**. The results are compared to identify the model that provides the best prediction accuracy.



```

Model': [
    'Decision Tree',
    'Random Forest',
    'KNN',
    'XGBoost'
],
'Accuracy': [
    accuracy_score(y_test, dt_pred),
    accuracy_score(y_test, rf_pred),
    accuracy_score(y_test, knn_pred),
    accuracy_score(y_test, xgb_pred)
]
])
model_results

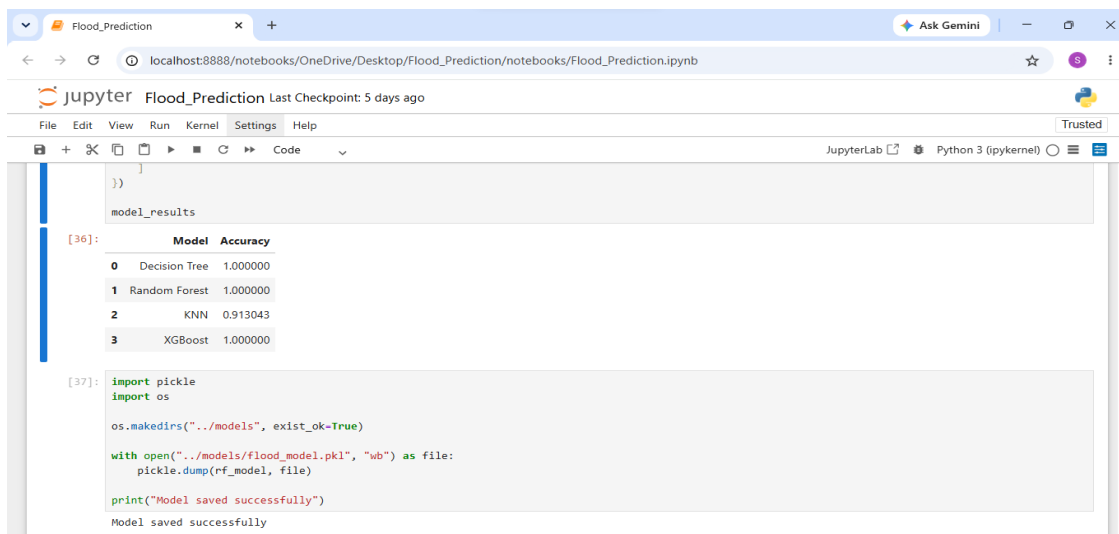
```

[36]:

	Model	Accuracy
0	Decision Tree	1.000000
1	Random Forest	1.000000
2	KNN	0.913043
3	XGBoost	1.000000

Activity 4.3: Save the Best Model

The best-performing model is selected based on the evaluation results and saved as a **Pickle (.pkl)** file. This saved model is later loaded into the Flask application to generate flood risk predictions.



```

})
model_results

[36]:
Model Accuracy
0 Decision Tree 1.000000
1 Random Forest 1.000000
2 KNN 0.913043
3 XGBoost 1.000000

[37]: import pickle
import os

os.makedirs("../models", exist_ok=True)

with open("../models/flood_model.pkl", "wb") as file:
    pickle.dump(rf_model, file)

print("Model saved successfully")
Model saved successfully

```

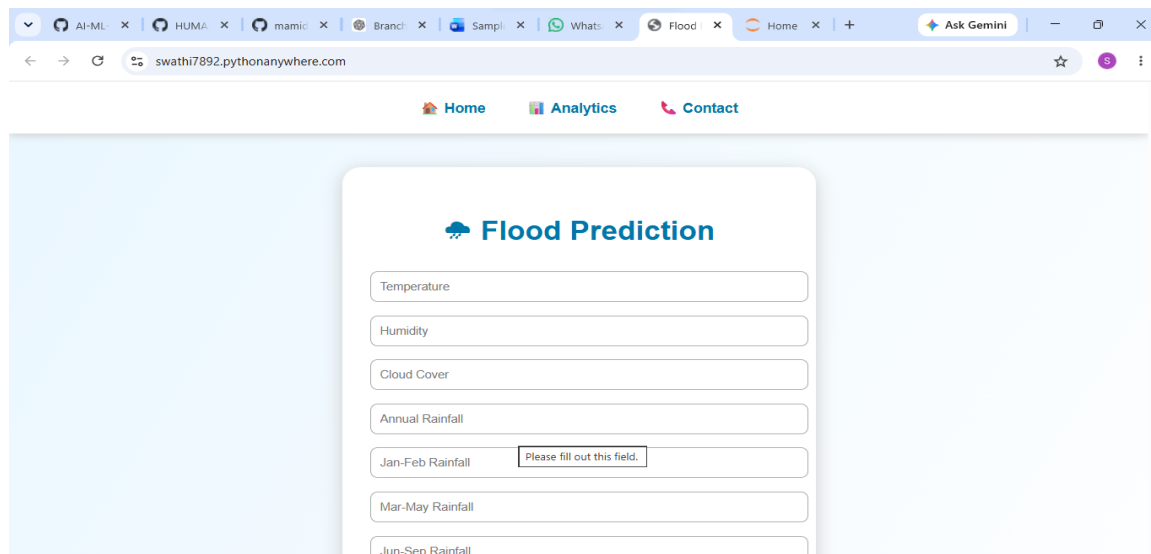
Milestone 5: Application Development and Deployment

Objective:

The objective of this milestone is to develop a user-friendly web application by integrating the trained machine learning model with Flask. The application allows users to enter weather-related parameters, predict flood risk, and access model analytics and emergency contact information. Finally, the application is tested and deployed for public access.

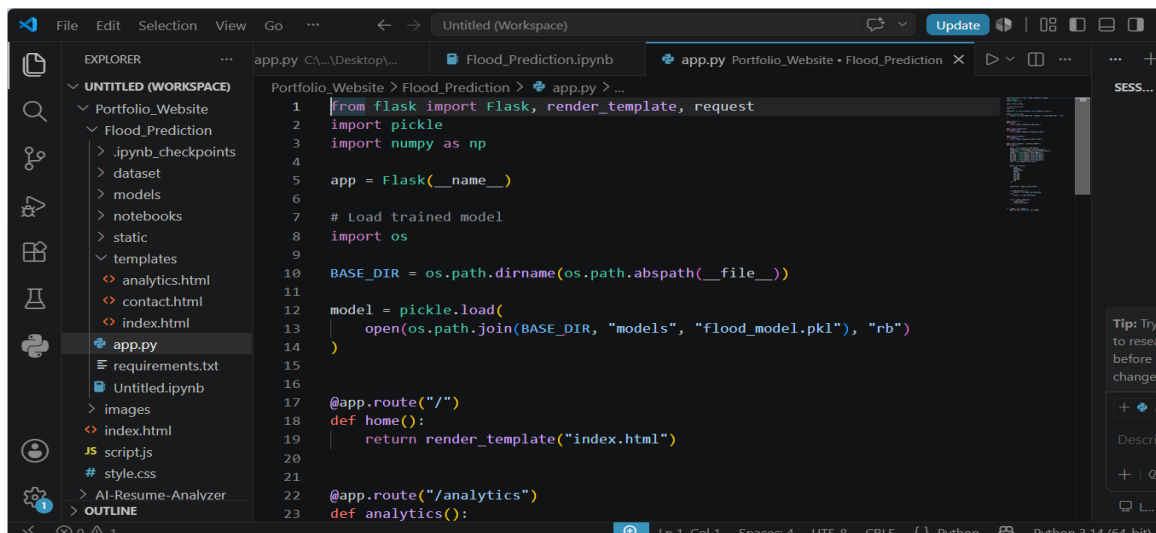
Activity 5.1: Develop the Web Interface

A responsive web interface is designed using **HTML and CSS** to provide an easy-to-use platform where users can enter weather parameters required for flood prediction.



Activity 5.2: Integrate the Machine Learning Model

The trained **flood_model.pkl** is integrated into the Flask application (app.py). User inputs are processed, and the model generates flood risk predictions that are displayed on the web page.

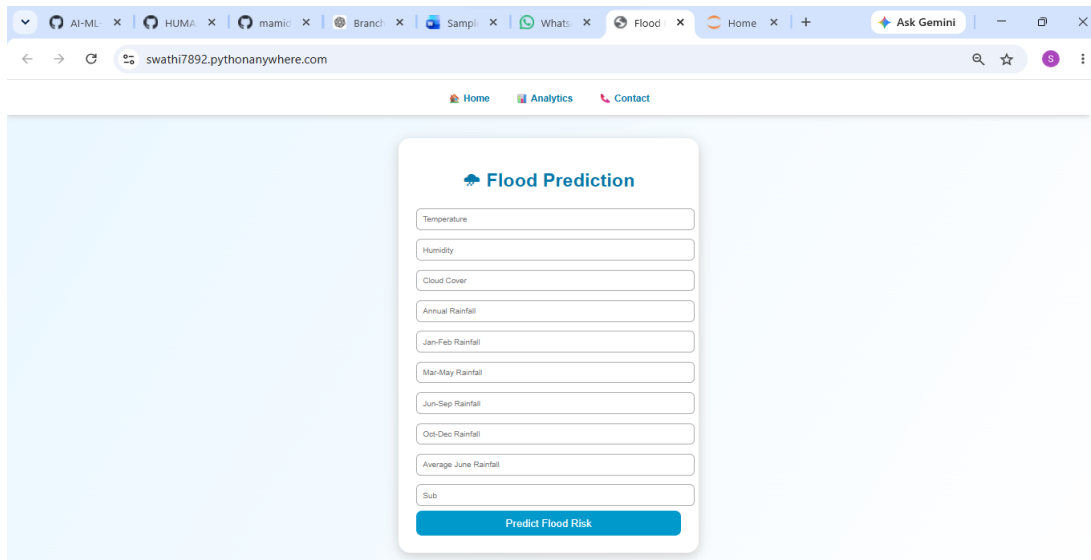


Activity 5.3: Test and Deploy the Application

The application is tested locally using the Flask development server to verify correct predictions and page navigation. After successful testing, the project is deployed on **PythonAnywhere**, making it accessible through a web browser.

Exploring the Website's Web Pages:

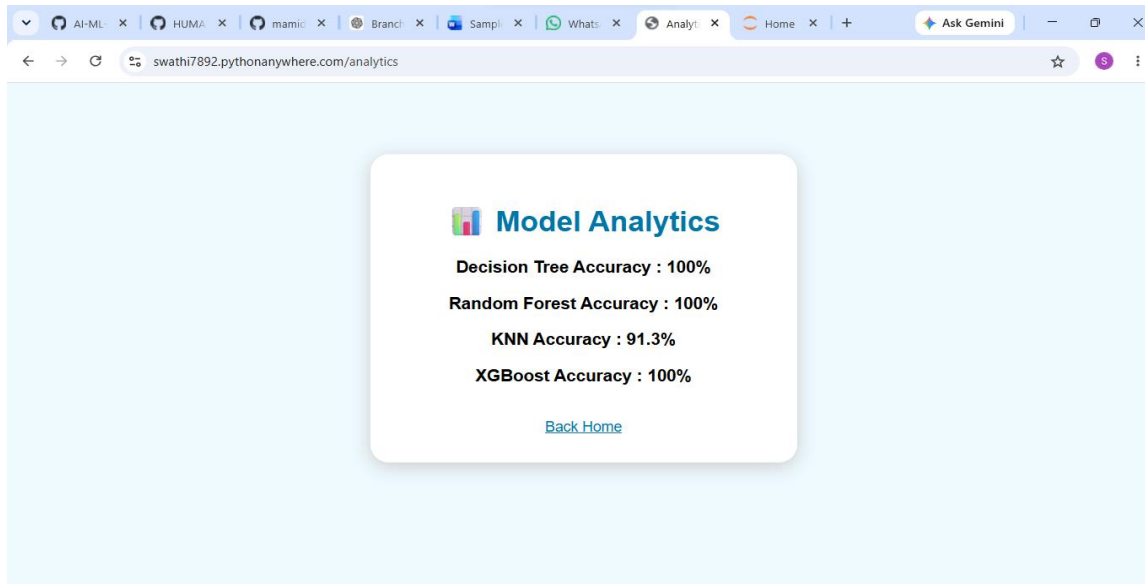
Home/Get Started Page:



The screenshot shows a web browser window with the URL `swathi7892.pythonanywhere.com`. The page has a light blue background. At the top, there are navigation links: Home, Analytics, and Contact. The main content area features a white box titled "Flood Prediction" with a blue icon of a cloud and rain. Inside this box, there are ten input fields for weather and rainfall data: Temperature, Humidity, Cloud Cover, Annual Rainfall, Jan-Feb Rainfall, Mar-May Rainfall, Jun-Sep Rainfall, Oct-Dec Rainfall, Average June Rainfall, and a Sub field. A blue button labeled "Predict Flood Risk" is positioned at the bottom of the form.

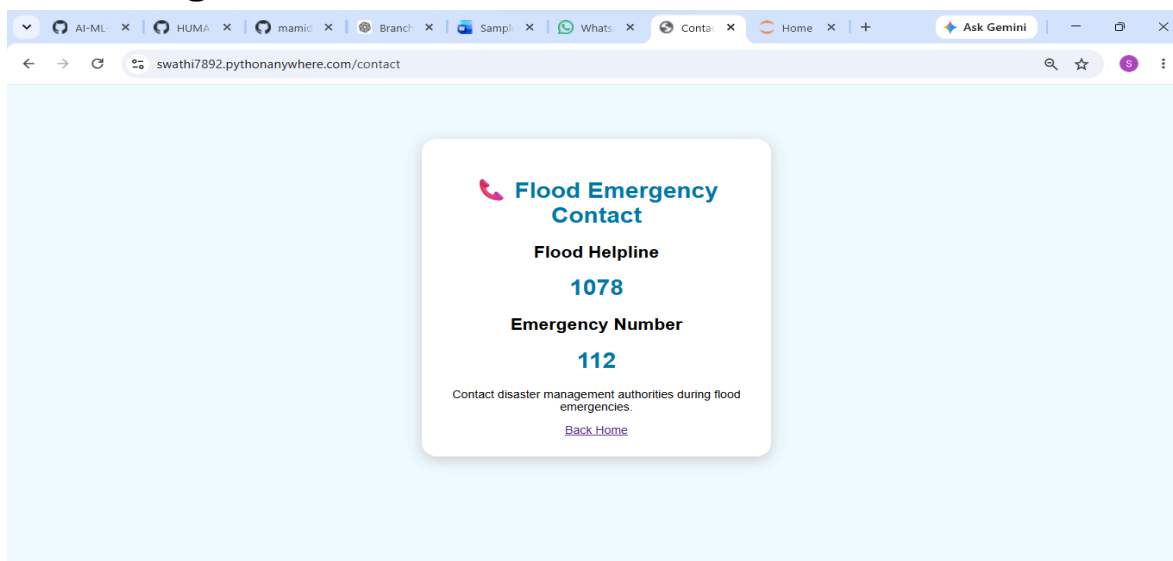
Description: The **Home/Get Started** page is the main interface of the Rising Waters application. It allows users to enter weather and rainfall parameters required for flood prediction. The page provides a simple and user-friendly design for easy navigation. Users can submit the entered values to receive instant flood risk predictions. It also includes navigation links to the **Analytics** and **Contact** pages.

Analytics Page:



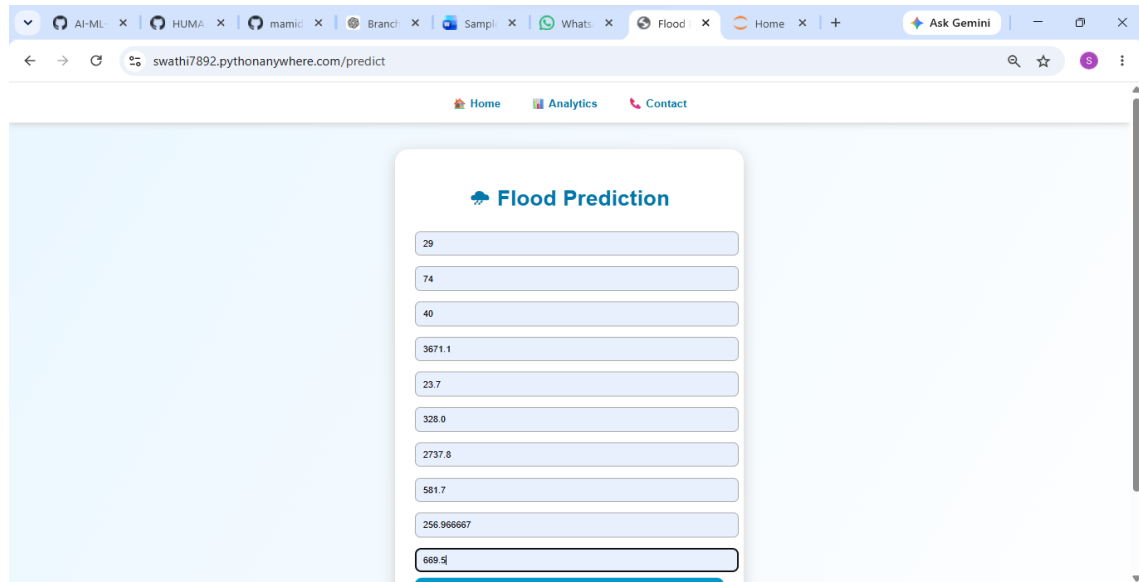
Description: The **Analytics** page displays the performance of the machine learning models used for flood prediction. It provides evaluation metrics such as accuracy and model comparison results to help users understand the effectiveness of the prediction system. The page presents the information in a clear and organized manner for easy interpretation.

Contact Page:

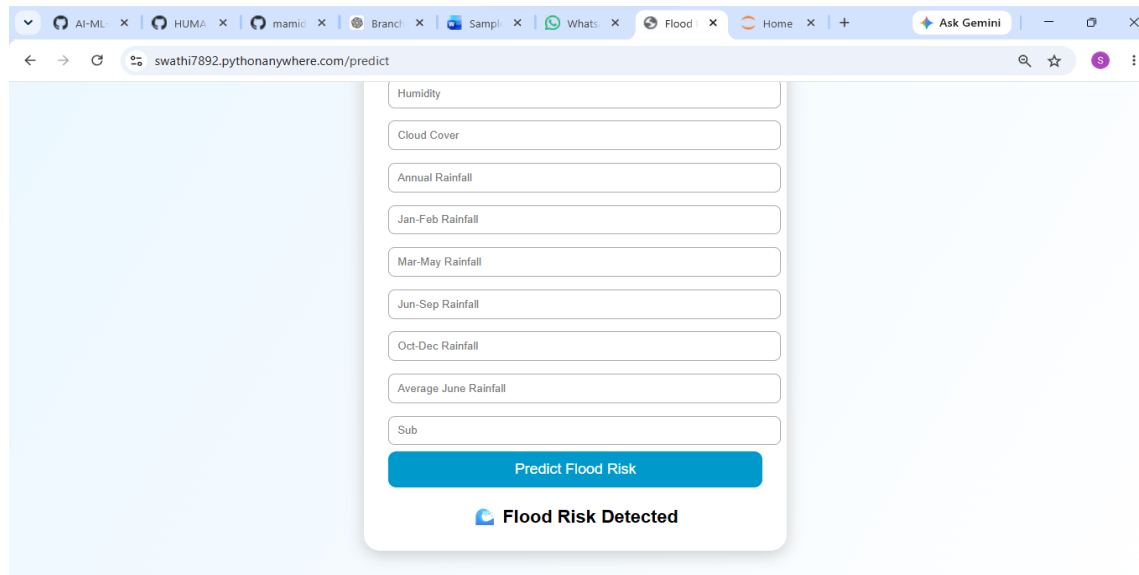


Description: The **Contact** page provides emergency flood helpline numbers and essential contact information for disaster management support. It enables users to quickly access assistance during flood-related emergencies. The page is designed with a simple layout to ensure important contact details are easily accessible.

Result 1: Flood Risk Detected



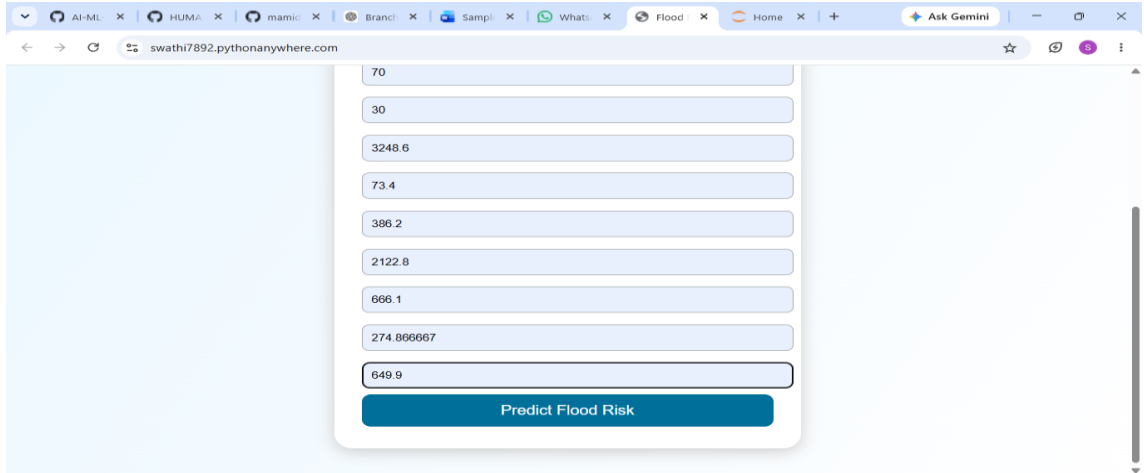
The screenshot shows a web browser window with the URL `swathi7892.pythonanywhere.com/predict`. The page has a navigation bar with links for Home, Analytics, and Contact. The main content area features a central white card titled "Flood Prediction" with a blue cloud icon. Inside the card, there are ten input fields, each containing a numerical value: 29, 74, 40, 3671.1, 23.7, 328.0, 2737.8, 581.7, 256.966667, and 669.5. The card is set against a light blue background.



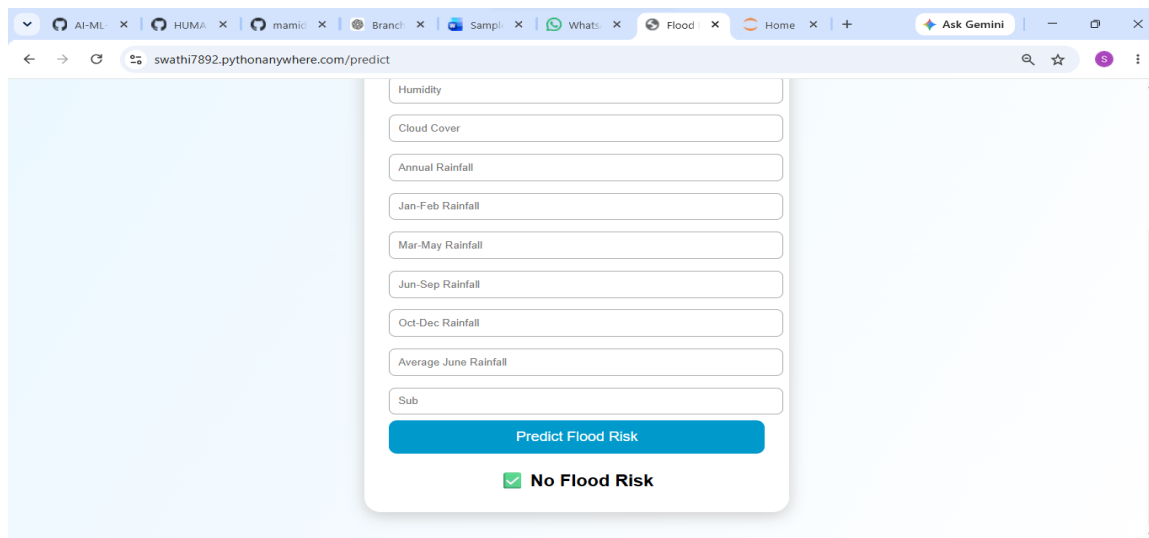
The screenshot shows the same web browser window after a prediction. The central white card now displays a list of weather and rainfall parameters in input fields: Humidity, Cloud Cover, Annual Rainfall, Jan-Feb Rainfall, Mar-May Rainfall, Jun-Sep Rainfall, Oct-Dec Rainfall, Average June Rainfall, and Sub. Below these fields is a prominent blue button labeled "Predict Flood Risk". At the bottom of the card, a blue cloud icon is followed by the text "Flood Risk Detected".

Description: The application predicts a **Flood** when the entered weather and rainfall parameters indicate a high risk of flooding. After clicking the **Predict Flood Risk** button, the trained machine learning model processes the input values and displays the prediction result. This helps users identify potential flood situations and supports early preparedness.

Result 2: No Flood Risk Detected



70
30
3248.6
73.4
386.2
2122.8
666.1
274.866667
649.9
Predict Flood Risk



Humidity
Cloud Cover
Annual Rainfall
Jan-Feb Rainfall
Mar-May Rainfall
Jun-Sep Rainfall
Oct-Dec Rainfall
Average June Rainfall
Sub
Predict Flood Risk
✓ No Flood Risk

Description: The application predicts **No Flood** when the input weather conditions do not indicate a significant flood risk. The prediction is generated instantly after processing the user-provided values through the trained machine learning model. This helps users understand that the current conditions are considered safe.

Conclusion:

The **Rising Waters – Flood Prediction System** was successfully developed to predict flood risk using historical weather and rainfall data. The project involved data collection, preprocessing, exploratory data analysis, machine learning model development, and deployment through a Flask web application. Multiple machine learning algorithms, including **Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost**, were trained and evaluated to identify the most effective model for flood prediction.

The final application provides a simple and user-friendly interface where users can enter weather-related parameters and receive instant flood risk predictions. By integrating machine learning with a web-based platform, the project demonstrates how intelligent technologies can support early flood prediction and disaster preparedness. Overall, the system provides an effective solution for flood risk assessment and highlights the practical application of machine learning in addressing real-world environmental challenges.